ИТМО

Программирование ПЛИС

TestBench

Власов Сергей Михайлович

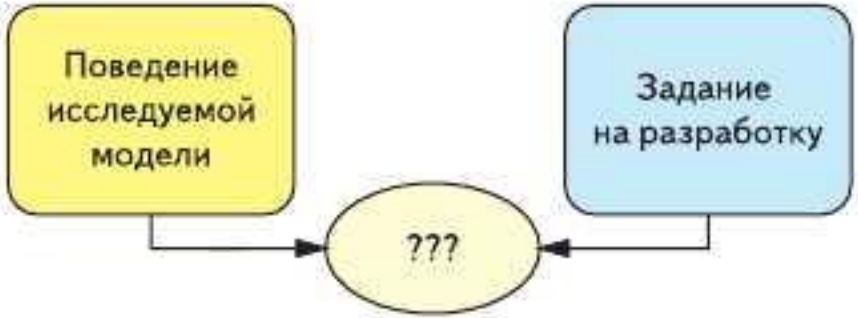
TestBench

Среда тестирования – это модуль на HDL, который используется для тестирования другого модуля, называемого тестируемое устройство (device under test, DUT). (Некоторые программы разработки называют тестируемый модуль unit under test, UUT.)

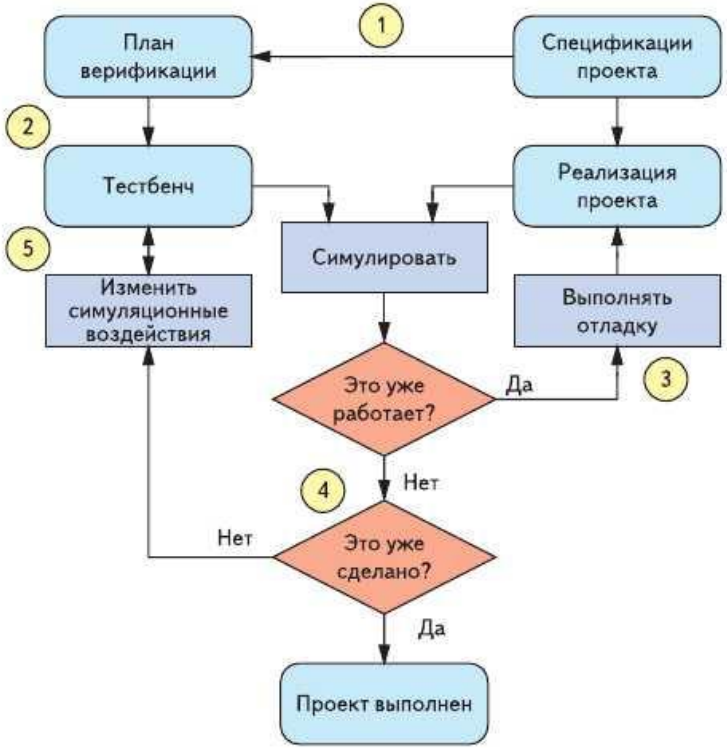
Среда тестирования содержит операторы для генерации значений, подаваемых на входы DUT и, в идеале, также и для проверки, что на выходе получаются правильные значения.

Наборы входных и желаемых выходных значений называются тестовыми векторами.

TestBench



TestBench


1
План верификации
Спецификации проекта
2
Тестбенч
Реализация проекта
5
Симулировать
Изменить симуляционные воздействия
Выполнять отладку
Это уже работает?
Да
3
Нет
4
Нет
Это уже сделано?
Да
Проект выполнен

TestBench Пример - Подходы

Шаг № 1. Делаем файл testbench с UART’ом. Создаем тактовую частоту из системной, путем деления системной частоты на счетчике. Делаем заглушку «передача на прием».
Шаг № 2. Передаем байт данных, принимаем этот же байт, визуально сравниваем переданный и принятый байты. Если они совпали, то на этом проверка завершается.

TestBench Пример - Подходы

Шаг № 1. Для начала введем в testbench две системных тактовых частоты: одну для приема, а другую для передачи. Соответственно сделаем и две бодовых частоты: одну для передатчика, другую для приемника. Теперь сформируем массив данных для передачи в виде файла. В testbench сделаем выдачу данных в передатчик из файла, а в приемнике сделаем запись данных в файл принимаемых данных. Дополнительно можно и в testbench сформировать функцию проверки переданных и принятых данных.

Шаг № 2. Делаем системные тактовые частоты равными. Проводим проверку тракта передача/прием. Результат сравниваем в файлах. Если совпали, переходим к следующему шагу.

Шаги № 3 и 4. Моделируем разбег частот генераторов системной тактовой частоты. Генератор «перекашиваем» в разные стороны. Уход частоты генератора – статический, временной и температурный. Здесь также необходимо учесть то, что делитель бодовой частоты тоже может давать неточное приближение к стандартной бодовой частоте.

Шаги № 5 и 6. В этом разделе мы будем проверять, формирует ли приемник бит ошибки по паритету. Выполняем проверку, аналогичную пунктам 3 и 4, но на стороне передачи делаем ошибку по паритету. Эту ошибку приемник должен показать в регистре состояний.

Шаги № 7 и 8. Теперь проверим тем же образом стоповый бит. На стороне передачи создаем неправильную посылку по стоповому биту. Эту ошибку приемник тоже должен показать в регистре состояний.

Шаг № 9. Мы проверили работу передатчика и приемника при идеальной линии связи. Теперь необходимо проверить работу приемника при сбойных импульсах в линии. Формируем «короткий» стартовый импульс в линии. Выберем одно из «крайних» значений тактового генератора и проводим тест. Приемник не должен принимать данные, но в регистре состояний может появиться соответствующий бит.

Шаг № 10. Осталось самое легкое: введем сбойные биты «внутрь» битового интервала данных на передаче. Если в приемнике сделан мажоритар, то он должен восстановить правильное значение данных.

TestBench


Ожидаемые результаты
Компаратор
Приложенные воздействия
DUT — исследуемое устройство
Полученные результаты

TestBench


Приложенные воздействия
Образец
Ожидаемые результаты
DUT — исследуемое устройство
Полученные результаты
Компаратор

TestBench


Генератор воздействий
Образец
DUT — исследуемое устройство
Компаратор

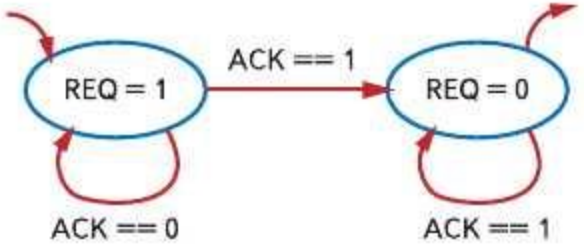
TestBench


Блок
результатов
тестирования
Генератор
воздействий
Коллектор
охвата
тестированием
DUT —
исследуемое
устройство

TestBenchs

```
Type STATE_TYP is (…, MAKE_REQ, RELEASE, …);
Signal STATE, NEXT_STATE: STATE_TYP;
…
COMB: process (STATE, ACK)
begin
   NEXT_STATE <= STATE;
   case STATE is
…
   when MAKE_REQ =>
      REQ <= '1';
      if ACK = '1' then
         NEXT_STATE <= RELEASE;
      end if;
   when RELEASE =>
      REQ <= '0';
      if ACK = '0' then
         NEXT_STATE <= …;
      end if;
   end case;
end process COMB;

SEQ: process (CLK)
begin
   if CLK'event and CLK = '1' then
      if RESET = '1' then
         STATE <= …;
      else
         STATE <= NEXT_STATE;
      …
      end if;
   end if;
end process SEQ;
```




```
process
   begin
         ...
      REQ <= '1';
         wait untill ACK = '1'
   ...
REQ
```

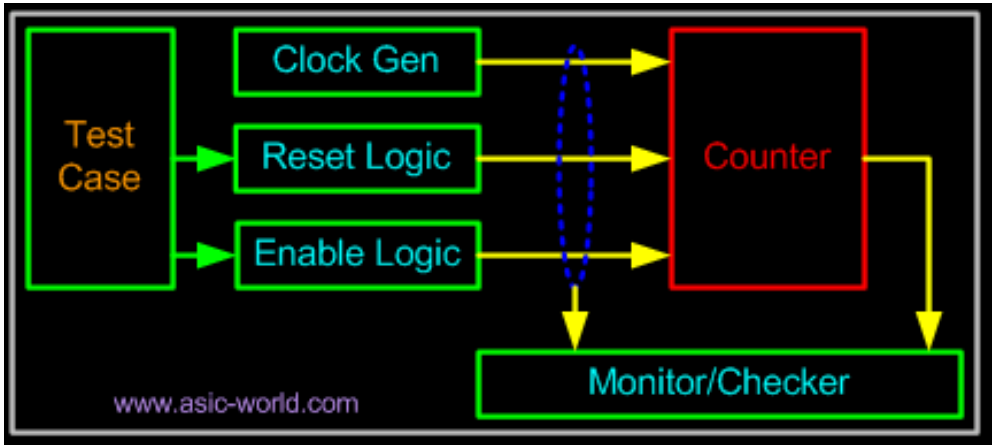
TestBench – пример.

```
module counter (clk, reset, enable, count);
input clk, reset, enable;
output [3:0] count;
reg [3:0] count;

always @ (posedge clk)
if (reset == 1'b1) begin
   count <= 0;
end else if ( enable == 1'b1) begin
   count <= count + 1;
end

endmodule
```

TestBenchs – Пример. План тестирования



TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps // `timescale <time_unit>/<time_precision>
module counter_tb;

endmodule
```

```
'timescale  - s, ms, us, ns, ps, fs
$printtimescale
$time
$realtime
$timeformat
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;

reg clk=0, reset, enable;
wire [3:0] count;

endmodule
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;
reg clk=0, reset, enable;
wire [3:0] count;

counter U0 (  .clk     (clk),
              .reset   (reset),
              .enable  (enable),
              .count   (count));

         // .inst port (couter_tb port)


endmodule
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;
reg clk=0, reset, enable;
wire [3:0] count;

counter U0 (.clk(clk), .reset(reset), .enable(enable), .count(count));

initial
begin
 $display ("##################################################");
 $display ("Start TestBench");

 clk = 0; reset = 0; enable = 0;
end

endmodule
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;
reg clk=0, reset, enable;
wire [3:0] count;

counter U0 (.clk(clk), .reset(reset), .enable(enable), .count(count));

initial
begin
 $display ("##################################################");
 $display ("Start TestBench");
 clk = 0; reset = 0; enable = 0;
end

always
  #5 clk = !clk;
endmodule
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;
reg clk=0, reset, enable;
wire [3:0] count;

counter U0 (.clk(clk), .reset(reset), .enable(enable), .count(count));

initial
begin
 $display ("####################################################");
 $display ("Start TestBench");
 clk = 0; reset = 0; enable = 0;
end

always #5 clk = !clk;

initial begin // Ход симуляции
    #5 $finish;
end

endmodule
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;
reg clk=0, reset, enable;
wire [3:0] count;

counter U0 (.clk(clk), .reset(reset), .enable(enable), .count(count));

initial begin
  $dumpfile ("counter.vcd");
  $dumpvars(0, counter_tb);
end

initial begin
 $display ("##################################################");
 $display ("Start TestBench");
 clk = 0; reset = 0; enable = 0;
end

always #5 clk = !clk;

initial begin // Ход симуляции
    #5 $finish;
end

endmodule
```

TestBenchs – Пример. Начало

```
`timescale 1 ps/ 1 ps
module counter_tb;
reg clk=0, reset, enable;
wire [3:0] count;

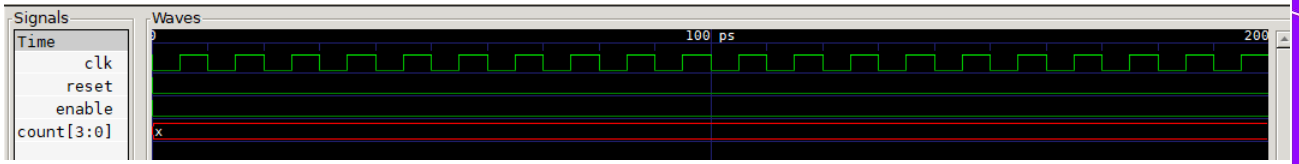
counter U0 (.clk(clk), .reset(reset), .enable(enable), .count(count));

initial begin
  $dumpfile ("counter.vcd");
  $dumpvars(0, counter_tb);
end

initial begin
 $display ("##################################################");
 $display ("Start TestBench");
 clk = 0; reset = 0; enable = 0;
end

always #5 clk = !clk;

initial begin // Ход симуляции
    #200 $finish;
end
// Add code here <-----------------
endmodule
```



TestBenchs – Пример. Симуляция reset

```
event reset_done; // Add code here <-----------------
event reset_enable;
```

TestBenchs – Пример. Симуляция reset

```
event reset_done; // Add code here <-----------------
event reset_enable;

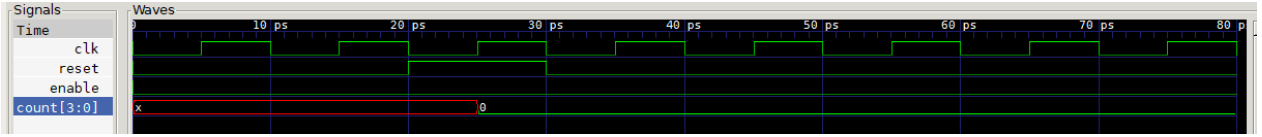
initial // Управление reset
forever begin                       // Бесконечный цикл
    @ (reset_enable);               // Ждем события Reset Enable
    @ (negedge clk)                 // После события Reset Enable ждем следующий падающий фронт clk
    $display ("Applying reset");
    reset = 1;                      // Задаем состояние линии reset
    @ (negedge clk)                 // Ждем один такт
    reset = 0;                      // Задаем состояние линии reset
    $display ("Came out of Reset");
    -> reset_done;                  // Вызываем событие – Reset Done
end
```

TestBenchs – Пример. Симуляция reset

```
event reset_done; // Add code here <-----------------
event reset_enable;

initial begin // Ход симуляции
    #10 -> reset_enable;
    @ (reset_done);
    @ (negedge clk);
    #10 $finish;
end

initial // Управление reset
forever begin
    @ (reset_enable);
    @ (negedge clk)
    $display ("Applying reset");
    reset = 1;
    @ (negedge clk)
    reset = 0;
    $display ("Came out of Reset");
    -> reset_done;
end
      // Add code here 2 <-----------------
```

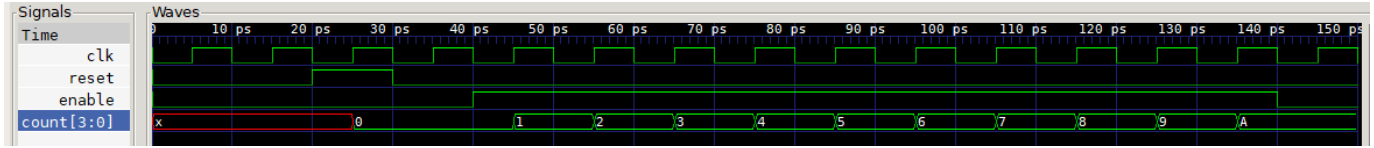


TestBenchs – Пример. Симуляция enable

```
initial begin // Ход симуляции
    #10 -> reset_enable;
    @ (reset_done);
    @ (negedge clk);

    enable = 1;
    repeat (10)
    begin
    @ (negedge clk);
    end
    enable = 0;

    #10 $finish;
end
```



TestBenchs – Пример. Выход

```
event terminate_sim; // <---------

// Окончание tesetbench
initial
@ (terminate_sim)  begin
 $display ("Terminating simulation");
 $display ("#################################################");
 #1 $finish;
end
```

TestBenchs – Пример. Выход

```
event terminate_sim; // <---------

initial begin // Ход симуляции
    ...
    ...
    #10 -> terminate_sim; // <---------
end

// Окончание tesetbench
initial
@ (terminate_sim)  begin
 $display ("Terminating simulation");
 $display ("#########################################################");
 #1 $finish;
end
```

TestBenchs – Пример. Терминал

```
##################################################
Start TestBench
VCD info: dumpfile counter.vcd opened for output.
Applying reset
Came out of Reset
Terminating simulation
##################################################
```

TestBenchs – Пример. Эталонная модель

```
// эталонная модель
reg [3:0] count_compare;
reg dut_error; // Незабываем добавить обнуление переменной в блоке старта
```

TestBenchs – Пример. Эталонная модель

```
// эталонная модель
reg [3:0] count_compare;
reg dut_error;

always @ (posedge clk)
if (reset == 1'b1)
 count_compare <= 0;
else if ( enable == 1'b1)
 count_compare <= count_compare + 1;
```

TestBenchs – Пример. Эталонная модель

ИТМО

```
// эталонная модель
reg [3:0] count_compare;
reg dut_error;

always @ (posedge clk)
if (reset == 1'b1)
 count_compare <= 0;
else if ( enable == 1'b1)
 count_compare <= count_compare + 1;

// сравнение объекта испытаний с эталонной моделью
always @ (negedge clk)
if (count_compare != count) begin
  $display ("DUT ERROR AT TIME%d",$time);
  $display ("Expected value %d, Got Value %d", count_compare, count);
  dut_error = 1;
  #10 -> terminate_sim;
end
```

TestBenchs – Пример. Эталонная модель

ИТМО

```
// Окончание tesetbench
initial
@ (terminate_sim)  begin
 $display ("Terminating simulation");
 if (dut_error == 0) begin
   $display ("Simulation Result : PASSED");
 end
 else begin
   $display ("Simulation Result : FAILED");
 end
 $display ("################################################");
 #1 $finish;
end
```

TestBenchs – Пример. Эталонная модель

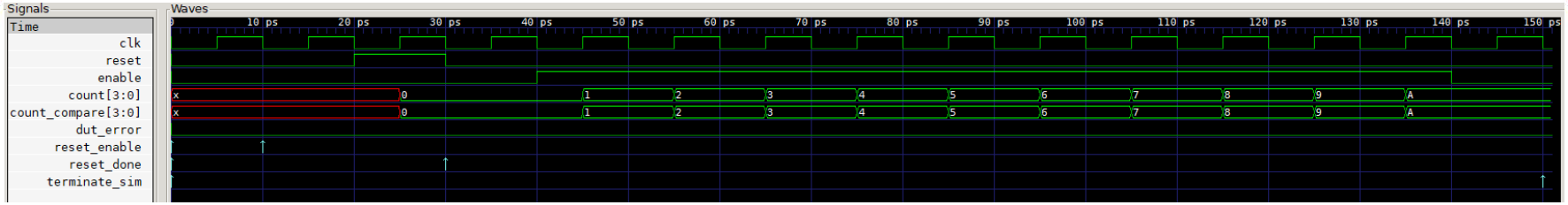
ИТМО

```
// Окончание tesetbench
initial
@ (terminate_sim)  begin
 $display ("Terminating simulation");
 if (dut_error == 0) begin
   $display ("Simulation Result : PASSED");
 end
 else begin
   $display ("Simulation Result : FAILED");
 end
 $display ("################################################");
 #1 $finish;
end
```

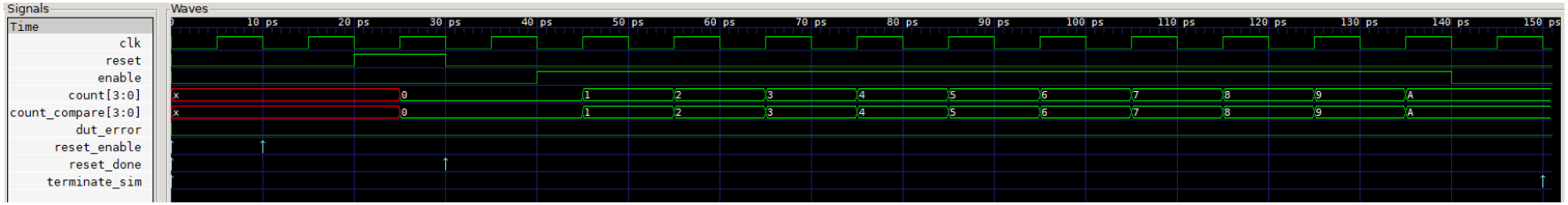
TestBenchs – Пример. Эталонная модель

```
################################################################
Start TestBench
VCD info: dumpfile counter.vcd opened for output.
Applying reset
Came out of Reset
Terminating simulation
Simulation Result : PASSED
################################################################
```

TestBenchs – Пример. Эталонная модель

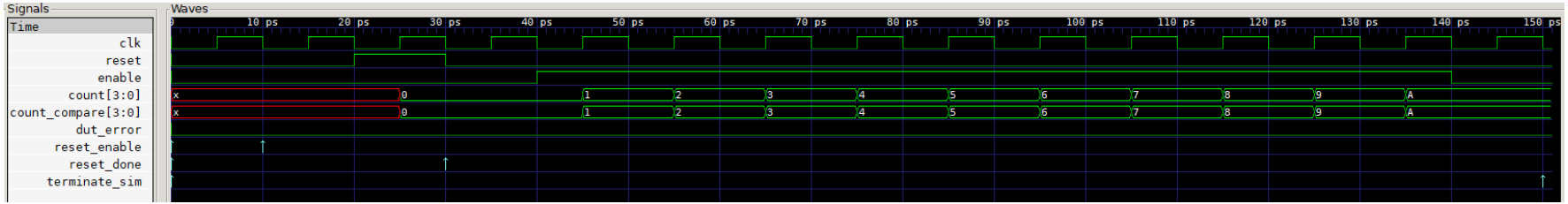


TestBenchs – Пример. Эталонная модель

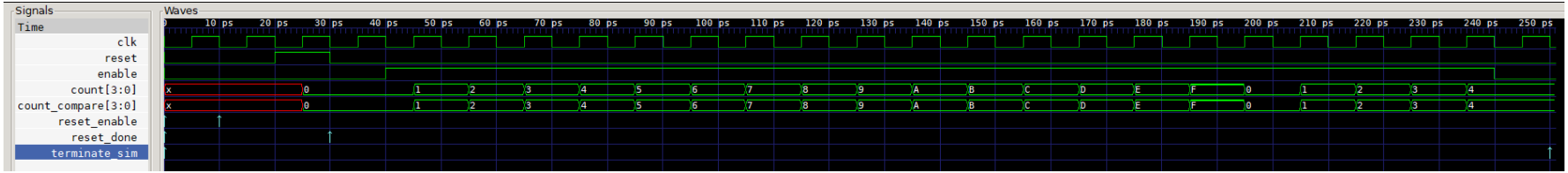



```
initial begin // Ход симуляции
    ...
    repeat (20)
    ...
end
```

TestBenchs – Пример. Эталонная модель




```
initial begin // Ход симуляции
    ...
    repeat (20)
    ...
end
```



TestBenchs – Пример. Случайные числа

```
initial begin // Ход симуляции
    ...
    #10 -> reset_enable;
    @ (reset_done);
    @ (negedge clk);

    fork        // внутри fork все выполняется одновременно
    repeat (50) begin
        @ (negedge clk);
        enable = $random;
        end
    repeat (50) begin
        @ (negedge clk);
        reset = $random;
    end
    join //end fork

    #10 -> terminate_sim;
end
```

TestBenchs – Пример. Случайные числа

